# DESIGN AND IMPLEMENTATION OF AUTOMATED ANSWER EVALUATION SYSTEM

*This project report is submitted to*

*Rashtrasant Tukadoji Maharaj Nagpur University*

*in the partial fulfilment of the requirement for the award of the degree of*

**Bachelor of Technology in Computer Technology**

*by*


**Ms. Shravani Chakinarapuwar (CT22081)**

**Mr. Rudraksh Mahurpwar (CT22027)**

**Mr. Ashwajit Surkar (CT22053)**

**Ms. Kesar Tandulkar (CT22083)**

**Mr. Akash Shelke (CTD22115)**


*Under the Guidance of*

**Mr. Bhushan A. Deshpande**

Assistant Professor

**2025-2026**


**DEPARTMENT OF COMPUTER TECHNOLOGY**

**KAVIKULGURU INSTITUTE OF TECHNOLOGY AND SCIENCE**

**RAMTEK, NAGPUR, MAHARASHTRA, INDIA-441106**

**KAVIKULGURU INSTITUTE OF TECHNOLOGY AND SCIENCE**
**RAMTEK, NAGPUR, M.S., INDIA-441 106**

**DEPARTMENT OF COMPUTER TECHNOLOGY**

# CERTIFICATE

This is to certify that the project report entitled **'Design and Implementation of Automated Answer Evaluation System'** carried out by Ms. Shravani Chakinarapuwar (CT22081), Mr. Rudraksh Mahurpwar (CT22027), Mr. Ashwajit Surkar (CT22053), Ms. Kesar Tandulkar (CT22083) and Mr. Akash Shelke (CTD22115) of B.Tech. fourth year of Computer Technology, during the academic year 2025-2026, in the partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology (Computer Technology)** offered by the **Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur.**

Mr. Bhushan A. Deshpande
**Guide**

Dr. Vilas P. Mahatme
**Head of the Department**

Dr. Avinash N. Shrikhande
**Principal**

Date:
Place: Ramtek

# ACKNOWLEDGEMENT

We are grateful to our respected guide **Mr. Bhushan A. Deshpande** for his kind, disciplined and invaluable guidance which inspired me to solve all the difficulties that came across during completion of project.

We express our special thanks to **Dr. Vilas P. Mahatme,** Head of the Department, for his kind support, valuable suggestion and allowing me to use all facilities that are available in the department during this project.

Our sincere thanks are due to **Dr. Avinash N. Shrikhande**, Principal, for extending all the possible help and allowing us to use all resources that are available in the Institute.

We are also thankful to our **Parents** and **Friends** for their valuable co-operation and standing with us in all difficult conditions.

Project-mates

# ABSTRACT

The rapid growth of digital education has created a pressing need for efficient and reliable methods of evaluating student performance. Traditional manual grading is time-consuming, subjective, and often inconsistent, leading to delays in providing feedback. Automated systems aim to address these challenges through the use of natural language processing and semantic analysis.

This project focuses on developing an automated answer evaluation system capable of assessing student responses with improved accuracy. The system leverages advanced language models to measure semantic similarity between student answers and reference solutions. By doing so, it ensures that grading is based on conceptual understanding rather than exact keyword matches.

The primary problem addressed by this project is the lack of scalable and unbiased grading mechanisms in educational institutions. Manual assessment often struggles to keep pace with the increasing number of students and diverse learning environments. The proposed solution bridges this gap by providing an automated, consistent, and transparent evaluation process.

At its core, the system computes semantic similarity scores using pre-trained models and generates rubric-based assessments. It also provides meaningful feedback to students, highlighting strengths and areas for improvement. This combination enhances both the grading process for teachers and the learning outcomes for students.

The application flow includes inputting the question, model answer, and student answer, followed by similarity calculation and score generation. The system then produces a detailed evaluation that mirrors rubric-based grading methods. Sample test cases validate the system's functionality, demonstrating its practical relevance and usability.

In conclusion, the project successfully establishes the feasibility of an automated evaluation system for descriptive answers. While tested on a limited scale, the results indicate significant potential for broader deployment in digital learning platforms. The study lays a foundation for future advancements in AI-driven education and assessment systems.

**KEYWORDS:** Automated Answer Evaluation, Natural Language Processing (NLP), Sentence-BERT (SBERT), Semantic Similarity, Rubric-based Scoring, Streamlit, Mistral 7B, SQLite

# CONTENTS


| | | |
|---|---|---|
| *Acknowledgment* | | i |
| *Abstract* | | ii |
| *Contents* | | iii |
| *Abbreviations* | | vi |
| *List of Figures* | | vii |
| *List of Tables* | | viii |
| **CHAPTER 1** | **INTRODUCTION** | **1-3** |
| | 1.1 Preamble | 1 |
| | 1.2 Motivation | 3 |
| | 1.3 Aim | 3 |
| | 1.4 Objectives | 3 |
| | 1.5 Organization of Report | 3 |
| **CHAPTER 2** | **Prior Art** | **4-6** |
| | 2.1 Semi-automated evaluation of long answer exams | 4 |
| | 2.2 Systems and methods for distilled BERT-based training model, for text classification | 4 |
| | 2.3 Method and system for intelligent identification and correction of questions | 5 |
| | 2.4 Automated Answers Evaluation and Scoring System and Method | 6 |
| **CHAPTER 3** | **LITERATURE REVIEW** | **7-8** |
| | 3.1 AI-Based Grading Systems and Applications in Education | 7 |
| | 3.2 Foundational NLP Techniques for Semantic Text Evaluation | 8 |

| | | |
|---|---|---|
| **CHAPTER 4** | **PROPOSED APPROACH AND SYSTEM ARCHITECTURE** | **9-17** |
| | 4.1 Proposed Approach | 9 |
| | 4.2 System Architecture | 12 |
| | 4.3 Module of the System | 15 |
| | 4.4 Algorithmic Design | 17 |
| | 4.5 LLM-based Rubric Evaluation and Score Aggregation | |
| **CHAPTER 5** | **TOOLS AND TECHNOLOGIES** | **18-23** |
| | 5.1 Programming Language | 18 |
| | 5.2 Libraries and Frameworks | 19 |
| | 5.3 Feedback Generation: Mistral 7B Instruct model | 22 |
| | 5.4 Database: SQLite | 23 |
| | 5.5 Development Environment | 23 |
| | 5.6 Version Control | 23 |
| **CHAPTER 6** | **IMPLEMENTATION** | **24-33** |
| | 6.1 System Setup and Installation | 24 |
| | 6.2 Database Schema Design | 24 |
| | 6.3 User Interaction and Interface | 27 |
| | 6.4 Core Functional Modules | 29 |
| | 6.5 Semantic Similarity and Rubric Evaluation | 30 |
| | 6.6 Implementation Screenshots | 32 |
| **CHAPTER 7** | **RESULTS AND DISCUSSION** | **34-38** |
| | 7.1 Application Flow | 34 |
| | 7.2 Sample Test Cases | 34 |

7.3 Grading Results 38

7.4 Feedback Results 38

**CHAPTER 8 CONCLUSIONS 39**

8.1 Limitations of the Study 39

8.2 Future Scope of Work 39

***References*** 40

# ABBREVIATIONS

| Abbreviations | Description |
|---|---|
| **NLP** | Natural Language Processing |
| **LLM** | Large Language Model |
| **SBERT** | Sentence Bidirectional Encoder Representations from Transformers |
| **API** | Application Programming Interface |
| **GUI** | Graphical User Interface |
| **UI/UX** | User Interface / User Experience |
| **SQL** | Structured Query Language |
| **DBMS** | Database Management System |
| **AI** | Artificial Intelligence |
| **ER** | Entity Relationship Diagram |
| **JSOM** | Javascript Object Notation |

# LIST OF FIGURES


| Figure | Title | Page |
|---|---|---|
| 4.1 | Workflow of the System | 10 |
| 4.2 | High Level Diagram of System Architecture | 12 |
| 5.1 | Tools and Technologies | 18 |
| 6.1 | ER Diagram | 27 |
| 6.2 | Add Question | 32 |
| 6.3 | Add Student | 33 |
| 6.4 | Submit Answer | 33 |
| 7.1 | Highly corrected answer result | 35 |
| 7.2 | Partially corrected answer result | 36 |
| 7.3 | Irrelevant answer result | 37 |

# LIST OF TABLES


| Table | Title | Page |
|---|---|---|
| 6.1 | Student Table | 25 |
| 6.2 | Questions Table | 25 |
| 6.3 | Submissions Table | 25 |
| 6.4 | Scores Table | 26 |

# CHAPTER 1
# INTRODUCTION

## 1.1 Preamble

Assessment is one of the most integral components of any education system, as it directly influences student learning outcomes, instructional design, and academic progress. Among the various forms of assessment, the evaluation of descriptive or subjective answers remains a particularly demanding task. Unlike multiple-choice or objective-type questions, which can be graded automatically with ease, descriptive answers require careful consideration of context, depth of understanding, logical flow, and relevance of content. This makes manual evaluation both time-consuming and labour-intensive, especially when dealing with large numbers of students.

A significant challenge in manual evaluation lies in its inherent subjectivity. Different evaluators may interpret the same answer differently, leading to inconsistencies in grading. Moreover, factors such as evaluator fatigue, time constraints, and varying levels of domain expertise can further affect the reliability of the assessment. In large-scale examinations or continuous learning environments, this inconsistency can impact fairness and hinder effective measurement of student performance.

Traditional attempts to automate subjective answer evaluation have primarily relied on keyword-based approaches or rule-driven techniques. While these methods provide some level of automation, they often fail to capture the semantic meaning of the student's response. For example, an answer that uses synonymous terms or a different phrasing but conveys the correct concept may be marked incorrectly if keyword overlap is missing. Conversely, a response containing the correct keywords without demonstrating conceptual clarity might be rewarded unjustly. These limitations highlight the inadequacy of surface-level evaluation methods.

Recent advancements in the fields of Artificial Intelligence (AI) and Natural Language Processing (NLP) have opened promising avenues to overcome these challenges. Modern language models, particularly those built on deep learning and transformer-based architectures, have demonstrated remarkable capabilities in understanding contextual meaning, identifying semantic similarity, and generating coherent human-like feedback. Such developments provide the foundation for designing intelligent systems that can

evaluate subjective answers more effectively and consistently, while also offering constructive feedback to learners.

The system envisioned in this work builds on these advancements to address the limitations of traditional evaluation methods. It is designed to compare student answers with reference responses in a meaningful way, not only by recognizing surface-level similarities but by analyzing the semantic alignment of ideas. In addition, the system aims to detect irrelevant or off-topic content and provide constructive feedback that highlights both the strengths and areas for improvement in a student's answer. By doing so, it seeks to combine the efficiency of automation with the fairness and insight expected from human evaluators.

In this context, automated answer evaluation is not merely a solution to the scalability problem but also an opportunity to enhance the overall learning experience. By integrating semantic understanding, consistency in grading, and personalized feedback, such systems can support educators, reduce their workload, and ensure fairer outcomes for students. Furthermore, they align with the growing shift towards digital learning platforms, online assessments, and large-scale education delivery models, making them increasingly relevant in today's academic and professional landscapes.

Beyond solving immediate challenges, the adoption of automated evaluation systems has the potential to transform how learning is measured and improved over time. By generating consistent insights across large volumes of responses, such systems can help identify common learning gaps, improve curriculum design, and provide educators with data-driven evidence to refine teaching strategies. This creates an ecosystem where evaluation is not just a measure of performance but also a tool for continuous educational enhancement.

Looking ahead, the role of intelligent evaluation systems is likely to become more significant as education increasingly embraces digital transformation. From classrooms to large-scale online courses, the demand for scalable, fair, and adaptive assessment methods will continue to grow. Automated systems that combine efficiency with meaningful feedback are therefore not only relevant today but also essential for shaping the future of education in a data-driven, technology-enabled world.

## 1.2 Motivation

The project is motivated by the need to reduce the time, effort, and inconsistencies involved in manual evaluation of descriptive answer sheets. It aims to provide a reliable system that can assess answers fairly and efficiently.

## 1.3 Aim

The aim is to create a reliable and efficient solution for evaluating subjective answers by combining automation with semantic understanding.

## 1.4 Objectives

- To design an automated system capable of evaluating descriptive student answers with consistency and fairness.
- To integrate semantic similarity techniques for assessing the conceptual alignment between student and reference answers.
- To implement a mechanism for identifying irrelevant or off-topic content in responses.
- To provide constructive feedback that highlights strengths and suggests improvements for learners.

## 1.5 Organization of Report

This report begins with the title page, certificate, acknowledgement, and abstract, followed by a detailed introduction that establishes the motivation, aim, and objectives of the project. Chapter two reviews prior art and existing patented methods for automated grading and document evaluation. Chapter three provides a literature review of research in AI-based grading systems and foundational NLP techniques such as Sentence-BERT. Chapter four describes the proposed approach and system architecture, including the workflow, modules, and algorithmic design. Chapter five outlines the tools and technologies used, including Python, Streamlit, Sentence-BERT, Mistral 7B, and SQLite. Chapter six details the implementation, covering database schema, user interaction, functional modules, and system screenshots. Chapter seven presents the results and discussion, including application flow, sample test cases, grading outcomes, and generated feedback. Finally, chapter eight concludes the report by summarizing achievements, addressing limitations, and proposing future scope, followed by references to all patents and research papers consulted.

# CHAPTER 2

# PRIOR ART

## 2.1 Semi-automated evaluation of long answer exams

## Application No.: - US11967253B2

This patent presents a computer-implemented system for semi-automated grading of essay or long-answer exams, aiming to reduce manual evaluation effort. The system receives an exam question along with a model answer, which is used as the reference for assessment. It identifies constructs, key phrases, or important points within the model answer to serve as grading criteria. When an examinee submits their answer, the system matches the content against the model answer constructs and assigns a preliminary grade. Portions of the examinee answer are marked with distinction markers, which indicate their relevance and priority in the context of the question. The graded answer sheet is then presented to a human evaluator, allowing verification or adjustment of scores. This hybrid approach ensures both speed and accuracy in grading. It is particularly beneficial for large classrooms or online assessment platforms. The system emphasizes consistency, fairness, and recognition of critical content. By combining automated grading with human oversight, it provides an efficient and reliable solution for subjective evaluations. It also allows for the tracking of student performance trends. Overall, it enhances the traditional evaluation process while minimizing evaluator workload.

## 2.2 Systems and methods for distilled BERT-based training model, for text classification

## Application No.: - US11967253B2

This patent presents a method for creating a smaller, efficient model from a large pre-trained BERT model through knowledge distillation. The large BERT model acts as a teacher, providing outputs that guide the training of a smaller student model. Both teacher and student models use the same input data, allowing the student to learn to approximate the behavior of BERT. This approach significantly reduces training time while retaining performance similar to the original BERT model. It is particularly useful for text classification and natural language processing tasks. The student model can be deployed in resource-constrained environments such as mobile devices or edge systems. The method balances model efficiency and accuracy, making large-scale NLP applications

more practical. It also enables faster experimentation and deployment in production systems. By transferring knowledge from the teacher to the student, the method ensures high-quality predictions with lower computational cost. Overall, it provides a scalable and efficient solution for high-performance NLP tasks.

## 2.3 Method and system for intelligent identification and correction of questions

### Application No.:- US 11,508,251 B2

This patent describes a method and system that leverages artificial intelligence for the automatic identification and correction of questions in test papers. The system is designed to recognize the structure and content of various question types, including multiple-choice, descriptive, and image-based questions. A neural network model, such as a convolutional neural network (CNN), is trained on a large dataset of sample test papers to accurately detect the position and type of each question. Once the question is located, the system identifies different components of the question, including the question stem, the answer section, and any associated images or diagrams. It can recognize printed font characters in the question stem and handwritten characters in the answer portion, enabling precise interpretation of both typed and handwritten content. After recognition, the system automatically corrects answers based on the identified character information, ensuring accurate grading. This approach reduces the need for manual intervention, saving significant time and effort for educators. The method is capable of handling multiple question formats and layouts efficiently, adapting to the diversity of real-world test papers. By employing AI, the system improves the consistency and fairness of the evaluation process, eliminating subjective biases in grading. It can enhance the speed and reliability of assessments, making it suitable for large-scale testing environments. The neural network can be retrained to accommodate new question styles or variations in handwriting, providing flexibility for future applications. The system also integrates seamlessly with digital evaluation platforms, streamlining workflow for educational institutions. By combining recognition, validation, and correction, it offers a fully automated solution for intelligent question identification and assessment. Overall, this patent represents a robust, scalable, and adaptive approach to modernizing examination grading through AI technology.

## 2.4 Automated Answers Evaluation and Scoring System and Method Application No.: - US5180309A

This patent discloses an apparatus and method for automated evaluation and scoring of participant responses to questions that are selected pseudo-randomly from a question bank. Each question belongs to a specific category and has predefined answers stored in an answer file, which serves as a reference for scoring. The system is capable of awarding partial credit for answers that are nearly correct, using a tolerance mechanism to account for acceptable variations in characters or format. A perfect answer earns full credit, while a partially correct answer receives a fraction of the full score, ensuring fair assessment. Questions are pseudo-randomly selected using a number generator seeded by the system clock, which ensures varied testing sequences for each participant. Participants select a category, and the system advances to the next question within that category, maintaining a structured assessment flow. Questions are displayed on a system interface, and participants submit their responses via a keyboard or similar input device. This automated process ensures consistent scoring across participants and minimizes manual grading errors. The method is suitable for large-scale testing environments, supporting numerous participants simultaneously. By integrating partial credit scoring, the system better reflects a participant's knowledge and understanding. The approach allows for efficient and timely evaluation of responses without compromising accuracy. It can also store and track performance data for analysis or reporting purposes. The system provides a comprehensive, reliable, and scalable solution for automated assessment in educational, professional, or certification contexts. Overall, this patent establishes a robust framework for fair, fast, and accurate automated scoring in various testing environments.

# CHAPTER 3
# LITERATURE REVIEW

## 3.1 AI-Based Grading Systems and Applications in Education

**Alexandra Gobrecht, Felix Tuma, (2024)** "Beyond Human Subjectivity and Error: A Novel AI Grading System" This paper introduces a novel automatic short answer grading (ASAG) system that aims to reduce human subjectivity and error in evaluating open-ended exam responses. Traditionally, grading such responses is a time-intensive task with significant implications for student outcomes. The proposed system utilizes a fine-tuned open-source transformer model, trained on a diverse dataset of exam answers drawn from multiple university disciplines. Initial evaluations showed high accuracy, even on unseen course material, indicating strong generalization capability. In a comparative study against certified human domain experts using historical grading data as ground truth, the model's performance was more consistent, achieving a 44% lower median absolute error than human re-graders. This highlights the potential for AI to deliver fairer and more scalable assessments in higher education, surpassing the limitations of multiple-choice systems and traditional human grading methods.

**Su-Youn Yoon (2023)** "Short Answer Grading Using One-shot Prompting and Text Similarity Scoring Model" This study presents an automated short answer grading (ASAG) model that delivers both analytic scores for sub-questions and a holistic overall score, enhancing interpretability and feedback quality. Recognizing that short answer questions often contain multiple components, the model identifies relevant text spans for each sub-question, enabling educators to understand how specific parts of a response influence scoring. To overcome the high cost of manual annotation, the approach combines large language model (LLM)-based one-shot prompting with a domain-adapted text similarity scoring model trained on a small annotated dataset. The model achieved promising performance, with an accuracy of 0.67 and a quadratic weighted kappa of 0.71. It emphasizes correctness and content coverage over writing quality, using justification keys to highlight the specific parts of student answers that contributed to the scores. This transparency allows educators to verify the fairness of the automated scoring and provides actionable feedback to students, demonstrating that effective ASAG models can be built with minimal data while maintaining interpretability and reliability.

**Ravinder Saharan, Rahul Kr. Chauhan, (2020)** "Automated Content Grading Using Machine Learning" This research investigates the use of machine learning techniques to automate the grading of theoretical exam answers, particularly in technical courses where manual grading is labor-intensive and prone to bias. The proposed system uses a combination of Bag of Words (BoW), term frequency–inverse document frequency (tf-idf), centroid vectors, and various lexical and semantic features to model and assess student responses. Training data was manually collected from exams taken by graduating students, enabling the development and evaluation of multiple machine learning models. The results demonstrated that content-specific grading in technical domains benefits from predictable answer structures and terminology, allowing algorithmic models to effectively assess relevance and correctness. The study also addresses challenges such as writing variability and domain-specific vocabulary, and highlights the role of OCR preprocessing in converting handwritten scripts into analyzable text. Overall, the findings confirm that traditional machine learning methods can deliver reliable and scalable grading performance comparable to human evaluators, while significantly reducing grading effort.

## 3.2 Foundational NLP Techniques for Semantic Text Evaluation

**Nils Reimers, Iryna Gurevych (2019)** "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" This paper introduces Sentence-BERT (SBERT), an extension of the BERT model designed to produce semantically meaningful sentence embeddings for efficient comparison in natural language processing tasks. Traditional BERT models require each sentence pair to be jointly processed, resulting in high computational costs, especially for tasks like semantic similarity search and large-scale clustering. SBERT addresses this by employing siamese and triplet network architectures to map sentences into a fixed-size embedding space, allowing quick similarity computations using measures such as cosine similarity. The authors demonstrate that conventional BERT-based sentence representations, such as averaging token embeddings or using the [CLS] token, perform poorly in semantic tasks, often worse than simpler models like GloVe. In contrast, SBERT significantly improves performance while reducing inference time from hours to seconds. Evaluated on multiple benchmarks, SBERT outperformed previous sentence embedding methods and proved highly effective for large-scale semantic search, retrieval, and transfer learning, making it a foundational tool for systems requiring fast and accurate sentence-level understanding.

# CHAPTER 4
# PROPOSED APPROACH AND SYSTEM ARCHITECTURE

## 4.1 Proposed Approach

The proposed system automates the grading of descriptive answers by comparing student responses to expert references using semantic understanding rather than keyword matching. It leverages Sentence-BERT to generate embeddings and uses cosine similarity to evaluate conceptual alignment between answers.

## Conceptual overview

The proposed system is based on the idea of automating the evaluation of descriptive student answers through semantic-level analysis. Traditional evaluation methods either rely on manual grading by teachers or employ computational approaches that compare responses using surface-level features such as keyword matching or pre-defined patterns. While keyword-based approaches provide partial automation, they fall short in measuring the depth of understanding or the contextual accuracy of an answer. In contrast, semantic-based evaluation focuses on the meaning conveyed by the student's response, rather than simply the presence or absence of certain words. This approach is particularly important in education, where students may express the same concept using different wording, examples, or sentence structures.

The foundation of the system lies in the use of pretrained language models such as Sentence-BERT, which can represent entire sentences as dense numerical vectors in a high-dimensional semantic space. These representations capture not just the surface form of words but also their contextual meaning. When both the student's answer and the reference answer are encoded into this vector space, the degree of similarity between them can be computed using cosine similarity. A high similarity score indicates that the student's response aligns well with the reference in terms of content and intent, even if the wording is different. This ability to measure semantic closeness is what distinguishes the proposed approach from conventional methods.

Keyword-based techniques, though simpler to implement, are limited in their effectiveness for evaluating descriptive answers. For instance, two students might provide equally valid explanations for a concept but use different terminologies; in such cases, a keyword-based system might unfairly penalize one of them. Similarly, a student may insert correct keywords in an otherwise incorrect or irrelevant context, which could

mislead the system into awarding undeserved marks. These shortcomings highlight the inability of keyword-driven methods to ensure fairness and accuracy in evaluation. Semantic similarity, on the other hand, overcomes these issues by focusing on the meaning behind the words rather than their literal form. This makes it a superior choice for contexts where conceptual understanding is the primary measure of correctness.

Another advantage of semantic evaluation is its robustness in handling linguistic variations. Students may use synonyms, paraphrasing, or even culturally specific expressions while still conveying the correct idea. A semantic-based system is more flexible in recognizing such variations, ensuring that students are rewarded for their understanding rather than penalized for stylistic differences. Moreover, this method also enhances inclusivity by accommodating diverse ways of expressing knowledge, which is particularly relevant in multilingual and multicultural educational environments.The overall concept of the proposed approach is to shift evaluation from a keyword-centric perspective to a meaning-centric one, thereby making the process more accurate, fair, and pedagogically valuable.

## Workflow of the System

The system processes student and expert-validated reference answers through a structured workflow that transforms raw input into fair scores and meaningful feedback. Each step as depicted in Fig 4.1 ensures accurate evaluation.

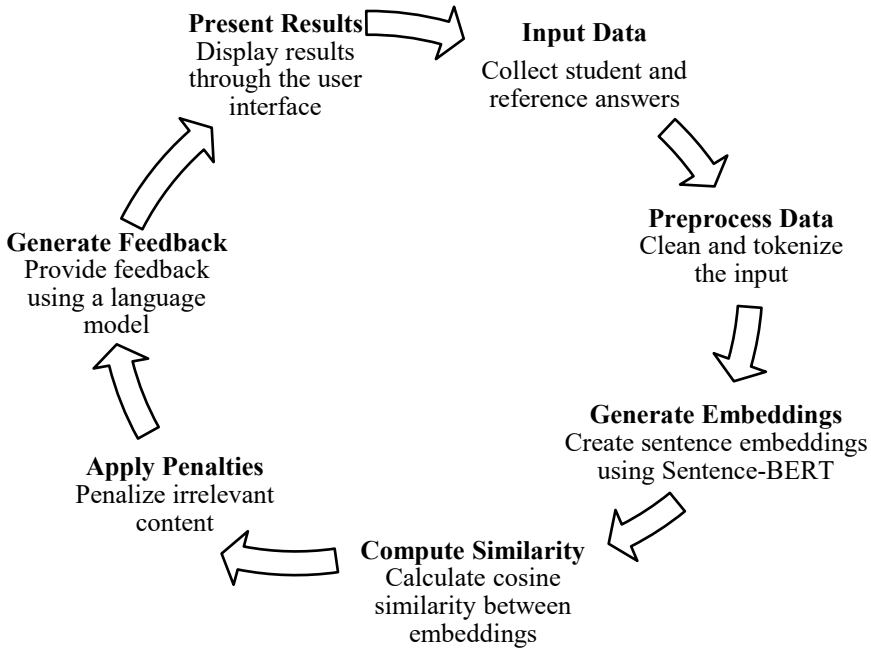


Fig 4.1 Workflow of the System

The first step in the workflow is preprocessing, where the student's answer and the reference answer are cleaned and segmented into sentences. This step ensures that the text is free of noise, such as unnecessary symbols or formatting issues, and that each sentence is treated as a distinct unit for analysis. Sentence-level tokenization is crucial because it allows the system to compare smaller, semantically coherent chunks of text rather than entire paragraphs, which might dilute the accuracy of similarity measurements.

Once preprocessing is complete, both the student's answer and the reference answer are passed to the embedding module powered by Sentence-BERT. In this stage, each sentence is transformed into a high-dimensional numerical vector that captures its semantic meaning. These embeddings form the basis for the subsequent similarity analysis. By converting text into a machine-readable format that preserves meaning, the system creates a reliable representation for comparing responses.

The next stage is the similarity computation module. Here, cosine similarity is calculated between each sentence embedding of the student's answer and the corresponding embeddings of the reference answer. This computation yields a similarity score that reflects how closely the student's statement aligns with the expert-provided reference. A higher cosine similarity value indicates stronger semantic overlap, while lower values suggest partial or poor alignment.

To ensure fairness and discourage irrelevant content, a penalty mechanism is integrated into the workflow. If the system detects sentences in the student's answer that fall below a predefined similarity threshold, those sentences are flagged as irrelevant or off-topic. A penalty is then applied to reduce the overall score, preventing students from gaining unfair advantage by including verbose but unrelated text. This mechanism ensures that the scoring remains balanced, rewarding relevant knowledge while penalizing deviations.

After scoring is finalized, the feedback generation module becomes active. In this stage, a large language model such as Mistral analyzes both the student's response and the evaluation results to generate qualitative feedback. The feedback highlights the strengths of the answer, identifies weaknesses, and provides suggestions for improvement. This dual role of assessment and guidance transforms the system into more than just a grading tool; it becomes an educational assistant that supports continuous learning. The final stage of the workflow is result presentation. The score and feedback are displayed to the user.

## 4.2 System Architecture

The Automated Answer Evaluation System features a Streamlit-based frontend where students submit answers. A Python backend handles processing, connecting to a Model Inference and Scoring module that uses Sentence-BERT and cosine similarity (with penalties) to assess answers. The Mistral language model provides rubric-based scoring and feedback. All results similarity scores, rubric evaluations, grades, and feedback are stored in SQLite and displayed on the frontend for review, overrides, and ongoing access. The system ensures seamless integration from input to evaluation, storage, and result presentation as depicted in the Fig 4.2.

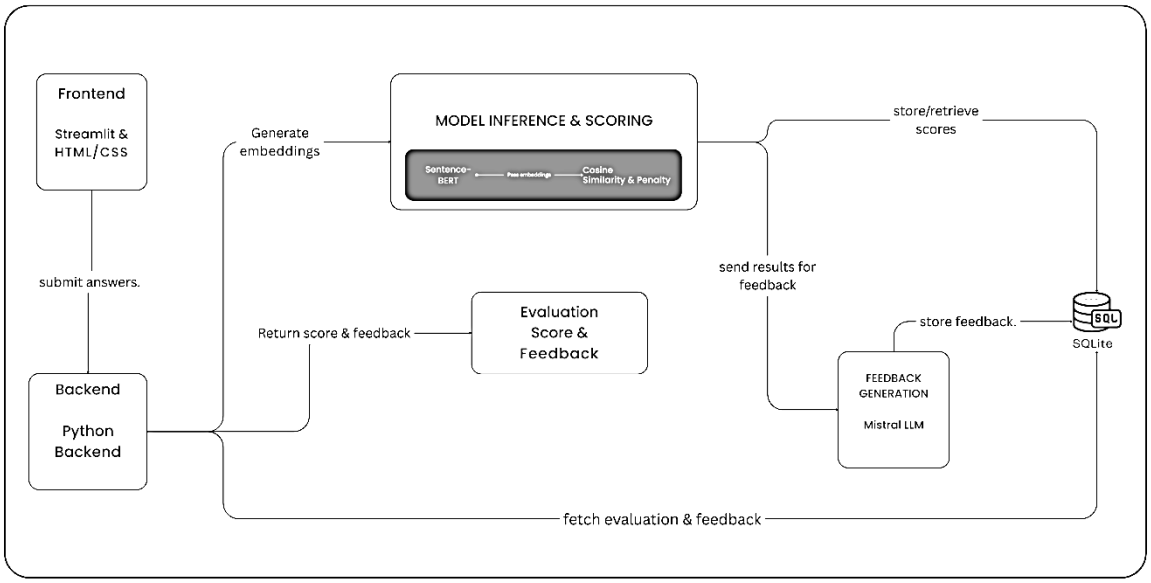


Fig 4.2 High Level Diagram of System Architecture

## 4.3 Modules of the System

## Input Module

The Input Module serves as the initial point of interaction between the user and the automated evaluation system. It is designed to capture both student responses and reference model answers in a structured manner, ensuring that the evaluation process begins with accurate and complete data. Beyond merely accepting text, this module is responsible for validating inputs to prevent incomplete or improperly formatted answers from entering the system. By providing a standardized interface, it ensures that all subsequent processing stages operate on consistent and reliable data. The module also plays a crucial role in maintaining fairness and transparency, as both student submissions and teacher-provided model answers are stored systematically for future reference.

## Preprocessing Module

Once data is acquired, the Preprocessing Module prepares it for semantic evaluation. Its primary function is to transform raw textual input into a structured and analyzable form. This involves operations such as normalization of text, which reduces variability caused by case differences or extraneous punctuation, and sentence tokenization, which divides answers into meaningful units suitable for sentence-level comparison. By organizing text into discrete sentences, the module facilitates a more nuanced and accurate assessment of content alignment with reference answers. Additionally, preprocessing ensures that irrelevant or noisy elements are minimized, thereby enhancing the quality of semantic analysis. This module, therefore, acts as a critical bridge between raw input and sophisticated embedding-based evaluation, enabling the system to assess answers reliably.

## Embedding Module

The Embedding Module is responsible for converting preprocessed sentences into semantic vector representations that capture contextual meaning. Rather than relying on surface-level keyword matches, this module utilizes advanced natural language understanding techniques to create dense numerical embeddings that encode the essence of each sentence. These embeddings enable the system to compare the meaning of student answers with model answers in a mathematically rigorous manner, allowing subtle differences in phrasing or expression to be evaluated accurately. The Embedding Module forms the core of semantic analysis within the system, as it provides a rich, multidimensional representation of textual content that is essential for precise similarity measurement. By abstracting sentences into meaningful vectors, it ensures that the evaluation process can distinguish between conceptually relevant content and irrelevant or off-topic material.

## Similarity Module

The Similarity Module leverages the embeddings produced in the previous stage to quantify the degree of alignment between student responses and reference answers. By calculating semantic similarity scores, this module provides an objective measure of how closely the meaning of each student sentence corresponds to that of the model answer. This evaluation is not limited to exact word matches but encompasses a broader understanding of semantic equivalence, allowing the system to recognize correct answers

expressed in varied language. Additionally, this module underpins the fairness and reliability of scoring by establishing thresholds for relevance, ensuring that only sufficiently aligned content contributes positively to the evaluation. It forms a critical link between semantic understanding and score computation, making it a pivotal component of the automated assessment pipeline.

## Penalty Module

The Penalty Module is designed to address content that is irrelevant, off-topic, or otherwise inconsistent with the model answer. When similarity scores fall below a defined threshold, this module proportionally reduces the overall evaluation score, thereby discouraging students from including unrelated or filler content. Its function is not punitive in nature but rather corrective, ensuring that the scoring system remains balanced and reflective of actual knowledge. By incorporating a penalty mechanism, the system maintains academic integrity and prevents superficial attempts to inflate scores through verbosity or tangential information. This module, therefore, enhances the credibility and rigor of the automated evaluation process by systematically adjusting scores to reflect content relevance accurately.

## Feedback Module

The Feedback Module provides qualitative insights into student performance, complementing the quantitative scores derived from similarity and penalties. Leveraging advanced language models, it analyzes both the content and structure of student answers to generate narrative feedback that highlights strengths, identifies weaknesses, and suggests specific areas for improvement. This module plays a crucial educational role, transforming the system from a mere scoring engine into a comprehensive learning tool. By offering actionable guidance, it enables students to understand not only where they performed well but also how they can refine their answers in future assessments. The Feedback Module, therefore, bridges the gap between automated evaluation and personalized learning, enhancing the system's value as a pedagogical resource.

## Database Module

The Database Module manages the storage and retrieval of all system data, including student submissions, model answers, embeddings, similarity scores, penalties, and feedback. By providing structured and persistent storage, it ensures that evaluation

records are maintained accurately and can be accessed for reporting, auditing, or further analysis. This module plays a critical role in data integrity, preventing loss or corruption of evaluation information and supporting scalability for multiple users. Additionally, the database enables longitudinal tracking of student performance, which can inform both pedagogical decisions and system improvement. As such, the Database Module acts as the backbone of the system, supporting reliable operation and facilitating future enhancements.

### Frontend Module

The Frontend Module constitutes the user interface of the system, enabling students and educators to interact seamlessly with the automated evaluation process. It presents a clean, accessible platform for answer submission, visualization of scores, and access to qualitative feedback. Beyond simple presentation, the frontend ensures usability by organizing information in a logical and interpretable manner, allowing users to understand evaluation outcomes effectively. It also supports transparency, as students can clearly see how their responses are assessed and where improvements are needed. The Frontend Module, therefore, is essential for user engagement and acts as the bridge between complex backend computations and the end-user experience.

## 4.4 Algorithmic Design

The algorithmic design of the automated answer evaluation system forms the backbone of its intelligent assessment capabilities. It integrates semantic understanding, relevance-based scoring, and personalized feedback to create a comprehensive evaluation framework. The design ensures that both quantitative and qualitative aspects of student answers are captured accurately while maintaining transparency and educational value.

### Semantic Similarity Algorithm

At the core of the system is the semantic similarity algorithm, which measures how closely a student's answer aligns with a reference model answer. The approach relies on vector embeddings to represent the semantic content of sentences in a high-dimensional space. Each sentence is transformed into a numerical vector that captures contextual meaning rather than mere lexical similarity. Semantic similarity between vectors is quantified using cosine similarity, which calculates the cosine of the angle between two vectors in the embedding space. Conceptually, if the vectors point in a similar direction,

the sentences are considered semantically close, indicating strong alignment with the reference answer. Mathematically, cosine similarity is expressed as:

**Cosine Similarity = (A · B) / (||A|| × ||B||)**

Where:

- A = Embedding vector of the student sentence
- B = Embedding vector of the model sentence
- A · B = Dot product of vectors A and B
- ||A|| and ||B|| = Magnitudes (lengths) of vectors A and B

This method allows the system to recognize conceptual equivalence even when sentences are expressed in different words or syntactic structures, ensuring a fair and nuanced evaluation of student answers.

## Penalty Mechanism

The penalty mechanism is conceptually designed to maintain the integrity and fairness of evaluation by addressing irrelevant or off-topic content. In this system, similarity scores derived from the semantic similarity algorithm are compared against predefined thresholds that represent minimum acceptable relevance. These thresholds are set based on expert guidance and empirical analysis of typical answer patterns, ensuring that the evaluation distinguishes between relevant content and filler material. When a student sentence fails to meet the threshold, it is flagged as less relevant, and a proportional penalty is applied to the overall score. This ensures that students are not rewarded for including unrelated content, while also encouraging concise and focused responses. By integrating penalties with similarity scores, the system balances quantitative scoring with qualitative judgment, maintaining academic rigor while reflecting true understanding.

## Feedback Generation Framework

Feedback generation is a crucial component that transforms the system from a scoring tool into a pedagogical aid. Conceptually, a large language model (LLM) is integrated to analyze student answers in the context of the model answer and similarity results. The LLM evaluates strengths, identifies weaknesses, and formulates actionable suggestions for improvement, creating a personalized learning experience. Feedback is not only important for guiding students toward better comprehension and expression but also enhances motivation and engagement, as learners receive constructive guidance rather than mere numerical scores.

## 4.5 LLM-based Rubric Evaluation and Score Aggregation

LLM-based Rubric Evaluation: In the proposed system, student answers are evaluated not only for semantic similarity but also using a rubric-based qualitative assessment. A large language model, Mistral 7B Instruct, is leveraged to:

- Score answers on Factual Accuracy, Completeness, Clarity, and Relevance (scale 0–5).
- Provide concise feedback highlighting strengths, weaknesses, and suggestions for improvement.

The rubric evaluation is designed to supplement the similarity score with human-like expert judgment, offering a richer, context-aware assessment.

Score Aggregation: To derive the final score, semantic similarity and rubric-based scores are combined with the following weighting scheme:

- 70% weight → similarity score
- 30% weight → rubric score

Additional safeguards prevent inflated scores: if similarity is very low but rubric scores are high, the rubric's influence is proportionally reduced. This ensures final scores reflect both alignment with the reference answer and qualitative evaluation dimensions such as clarity and completeness

# CHAPTER 5
# TOOLS AND TECHNOLOGIES

## 5.1 Programming Language

The primary programming language used in this system is Python. Python was chosen due to its widespread adoption in the fields of machine learning, natural language processing (NLP), and data science. Its extensive ecosystem includes numerous prebuilt libraries, frameworks, and tools that facilitate rapid development, experimentation, and deployment of complex algorithms. Python's syntax is highly readable and intuitive, making the codebase easier to maintain and extend. Furthermore, Python supports integration with APIs, databases, and web frameworks, which is essential for a system that combines backend processing, semantic analysis, LLM-based feedback generation, and a frontend user interface. Its versatility allows the development of both prototype and production-grade systems without the need for switching between multiple programming environments. The Fig 5.1 show the tools and technologies used graphically.

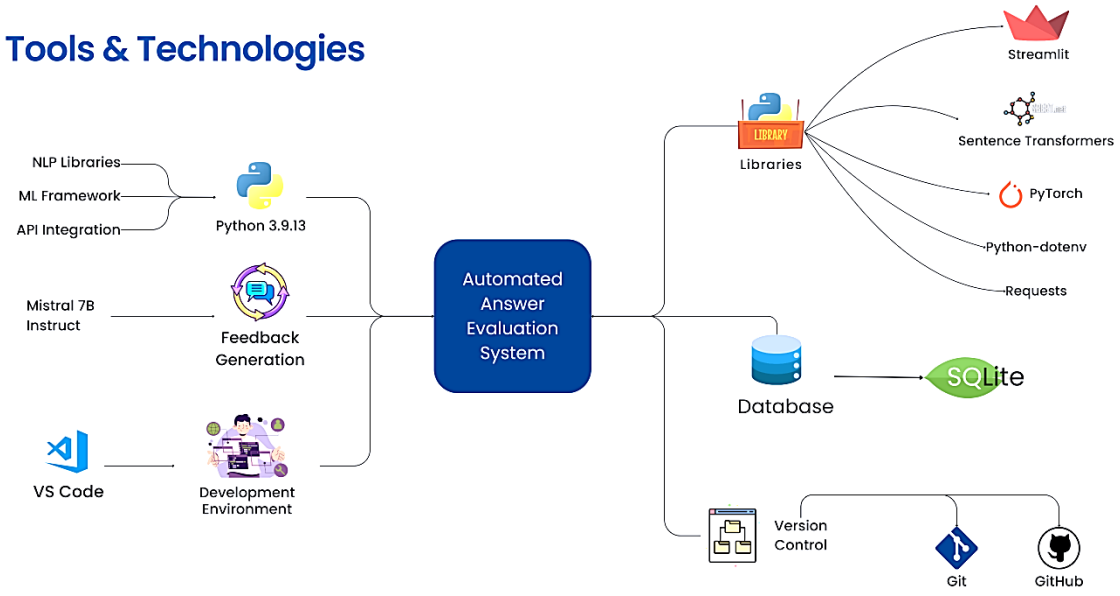


Fig 5.1 Tools and Technologies

## Python

In this project, Python 3.9.13 serves as the core programming language for implementing the backend logic of the automated answer evaluation system. It is employed to integrate and execute the Sentence-BERT model, which generates semantic embeddings for both student and reference answers, enabling precise similarity analysis. Python also facilitates

seamless interaction with the Mistral 7B Instruct API, allowing the system to generate automated and context-aware feedback for student responses. Additionally, Python manages all database operations with SQLite, ensuring efficient storage and retrieval of questions, answers, similarity scores, penalties, and feedback, thereby supporting the overall functionality and reliability of the system.

**Python Key features supporting this system**

- **Extensive NLP Libraries** Python provides access to advanced libraries such as NLTK, spaCy, and Sentence Transformers, which simplify text preprocessing, tokenization, and semantic embedding generation.
- **Machine Learning Frameworks** Libraries like PyTorch allow seamless integration of pre-trained models such as Sentence-BERT, enabling accurate semantic similarity computations.
- **Ease of API Integration** Python's requests library and other HTTP tools allow smooth communication with external services, including the Mistral 7B Instruct API for generating feedback.
- **Rapid Prototyping and Development** Python's high-level syntax and dynamic typing support fast experimentation, debugging, and iteration, which is critical for testing different evaluation and feedback algorithms.
- **Strong Community Support** Python has a vast community of developers and researchers in NLP and AI, providing extensive documentation, tutorials, and forums to assist in problem-solving and implementation challenges.
- **Integration with Databases** Python's built-in support for SQLite and other databases facilitates efficient storage, retrieval, and management of student responses, model answers, and evaluation results.
- **Frontend Support** Frameworks such as Streamlit allow developers to build interactive and user-friendly frontend interfaces directly using Python, bridging the gap between backend processing and end-user accessibility.

## 5.2 Libraries and Frameworks

The automated answer evaluation system integrates multiple Python libraries and frameworks to handle different aspects of NLP, frontend interaction, backend processing, and secure environment management. Each library contributes specifically to improving system accuracy, efficiency, and usability.

## Streamlit

Streamlit is an open-source Python framework for building interactive web applications, dashboards, and data visualization interfaces. It allows developers to convert Python scripts into shareable web apps with minimal code. In this system, Streamlit is used to create the interactive frontend where students submit descriptive answers and teachers provide reference answers. It also displays the evaluation results, including similarity scores, penalties, and generated feedback, in a user-friendly interface.

**Key features**

- Rapid development of web apps directly from Python scripts.
- Supports real-time updates, allowing dynamic display of evaluation results.
- Simple API for forms, buttons, and visualization widgets.
- Easy integration with backend Python logic.
- Accessible interface for non-technical users, enhancing usability in educational environments.

## Sentence Transformers

Sentence Transformers is a Python library built on top of PyTorch that provides pretrained models, including Sentence-BERT, to encode sentences into dense vector embeddings. These embeddings capture semantic meaning, enabling advanced NLP tasks like similarity measurement, clustering, and semantic search. The library is employed to generate embeddings for both student and reference answers. These embeddings are used to calculate semantic similarity, allowing the system to score answers accurately based on contextual alignment rather than just keyword matching.

**Key features**

- Pretrained models for fast and accurate embedding generation.
- Supports semantic similarity, clustering, and semantic search tasks.
- Easily integrates with PyTorch for GPU-accelerated processing.
- Allows sentence-level embeddings for granular analysis.
- Scalable for processing multiple answers simultaneously.

## PyTorch

PyTorch is a widely-used deep learning framework that provides flexibility, GPU support, and automatic differentiation for building and running neural network models. PyTorch underpins the deep learning operations of Sentence-BERT, enabling efficient

embedding computation. It also ensures that large-scale sentence processing can be performed quickly, supporting real-time evaluation for multiple students.

**Key features**

- Dynamic computation graphs for flexible model execution.
- GPU acceleration for faster processing of embeddings.
- Seamless integration with other Python libraries and frameworks.
- Robust support for neural network operations and tensor computations.
- Highly extensible, suitable for research and production-level applications.

## Python-dotenv

Python-dotenv is a Python library that allows secure management of environment variables through a .env file, avoiding hardcoding sensitive information in scripts. In this system, dotenv stores configuration parameters and API keys for services such as Mistral 7B Instruct, ensuring security and facilitating deployment across different environments.

**Key features**

- Secure storage of sensitive information like API keys.
- Easy to load environment variables into Python programs.
- Simplifies deployment across multiple environments.
- Prevents accidental exposure of credentials in code repositories.
- Lightweight and easy to integrate with existing Python projects.

## Requests

Requests is a Python library for sending HTTP requests to communicate with APIs and external web services. The library is used to interact with the Mistral 7B Instruct API, sending student responses and receiving detailed, context-aware feedback. It ensures smooth and reliable communication between the system and the LLM.

**Key features**

- Simple API for sending GET, POST, and other HTTP requests.
- Handles response parsing, headers, and authentication.
- Enables integration with external APIs and web services.
- Reliable error handling for robust communication.
- Lightweight and widely supported in Python applications.

## Regular Expression (re)

The re is a built-in module in python which provides support for regular expressions in Python, enabling pattern-based string processing, cleaning, and manipulation. It is extensively used for preprocessing student and reference answers, including removing unwanted characters, tokenizing text into sentences, and normalizing inputs for semantic embedding generation.

**Key features**

- Supports pattern-based string matching and manipulation.
- Enables advanced text preprocessing such as tokenization and cleaning.
- Allows search, split, and replace operations with high precision.
- Lightweight and part of Python's standard library.
- Essential for preparing text data for NLP tasks.

## 5.3 Feedback Generation: Mistral 7B Instruct model

The feedback generation module leverages the Mistral 7B Instruct model, a state-of-the-art large language model (LLM) designed to understand context and generate coherent, human-like text. Mistral is accessed through the Mistral API, which allows external applications to send input prompts and receive generated responses in real time.

In this system, Mistral 7B Instruct is used to generate detailed, narrative feedback for student answers. After the semantic similarity module identifies how closely a student's response aligns with the reference answer, the Mistral model interprets these results to provide qualitative guidance. This feedback highlights the strengths and weaknesses of each response, offers suggestions for improvement, and explains areas where the student could enhance their conceptual understanding. By integrating this advanced LLM, the system moves beyond numeric scoring to provide a personalized, educative experience.

**Applications of Mistral in the Project**

- Generates human-like, narrative feedback for textual inputs.
- Provides context-aware suggestions to improve student responses.
- Integrates seamlessly with similarity scores to ensure feedback relevance.
- Reduces the need for manual grading and written feedback.
- Supports scalable evaluation for multiple students simultaneously.

## 5.4 Database: SQLite

The system uses SQLite, a lightweight relational database management system, to store and manage all project-related data. Being file-based and requiring minimal setup, SQLite is well-suited for small- to medium-scale applications without the need for a full database server. It organizes entities like questions, model answers, student responses, similarity scores, penalties, and feedback in a structured schema for efficient data handling. Within the automated grading system, SQLite serves as the core for persistent data storage, maintaining inputs from students and teachers, intermediate results such as embeddings and similarity scores, applied penalties, and generated feedback. This durability ensures evaluation records are easily retrievable and auditable, supporting educators in reviewing past submissions, tracking performance, and analyzing learning trends. SQLite's lightweight design enables smooth integration with Python backend modules, minimizing overhead while ensuring reliable data management throughout the system.

## 5.5 Development Environment

The development environment prioritizes consistency, reproducibility, and maintainability by using virtual environment to create isolated Python environments. This isolation prevents conflicts between library versions across different projects, ensuring that dependencies for this project do not affect others on the same system.

Visual Studio Code (VS Code) is used for coding, testing, and debugging, offering features like syntax highlighting, IntelliSense, and version control. Combined with virtual environment, it helps manage dependencies and test modules independently, ensuring the project runs consistently across different machines.

## 5.6 Version Control

Version control is essential for tracking changes, collaboration, and managing project versions. The system uses Git for version control and GitHub for cloud-based repository management. Git records all code changes, enabling reversion, branching, and history tracking. GitHub provides a centralized platform for team collaboration, code sharing, reviews, and project management, ensuring smooth teamwork without conflicts and maintaining code integrity.

# CHAPTER 6
# IMPLEMENTATION

## 6.1 System Setup and Installation

The first step in implementing the automated answer evaluation system involves setting up a stable development environment. Python 3.9.13 was installed as the primary programming language because it is compatible with the libraries required for natural language processing, machine learning, and database management. A virtual environment (venv) was created to keep all project dependencies isolated, ensuring that they do not conflict with other Python projects on the system. This also provides a clean workspace for installing libraries and makes the project easily portable and reproducible across different machines. After the environment was prepared, all required Python libraries were installed, including Streamlit for the web interface, Sentence-Transformers and Torch for semantic similarity computations, Python-dotenv and Requests for API communication with the Mistral 7B model, and SQLite3 for backend database management. Following this, the SQLite database was initialized with the defined schema, which includes tables for students, questions, submissions, and scores. This setup ensures that the system is fully operational and ready to handle student data, process submissions, compute evaluations, and store results efficiently.

## 6.2 Database Schema Design

The database design serves as the foundation for the automated answer evaluation system. It is responsible for storing and managing all critical data, including student information, questions, submissions, and scores. The system uses SQLite as its database management system due to several reasons

- **Lightweight and self-contained** SQLite does not require a dedicated server, making it easy to set up and maintain.
- **Seamless Python integration** Being natively supported in Python, SQLite allows for smooth interaction with the backend using the sqlite3 module.
- **Suitable for prototyping and small-scale applications** Its simplicity and file-based nature make it ideal for academic projects and rapid development.
- **Portability** The entire database is stored in a single file, which can easily be transferred or backed up.

## Students Table.

Table 6.1 Illustrates the Students table, which represents the primary record of learners in the system. It contains fields such as student ID, roll number, and name, ensuring uniqueness and preventing duplication through constraints

Table 6.1 Student Table

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | Integer | Primary key, Auto increment | Unique identifier for a student |
| roll_number | Text | Unique, Not Null | Student's unique roll number |
| name | Text | Not Null | Student's full name |

## Questions Table

Table 6.2 show the Questions table, which stores all assessment items in the system.

Table 6.2 Questions Table

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | Integer | Primary key, Auto increment | Unique identifier for a question |
| question | Text | Not Null | The actual question text |
| reference_answers | Text | Not Null | Reference or model answer |

## Submissions Table

Table 6.3 Displays the Submissions table, which links students to their responses. It records submitted answers with timestamps and enforces the rule that each student can submit only once per question.

Table 6.3 Submissions Table

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | Integer | Primary key, Auto increment | Unique submission ID |
| student_id | Integer | Not Null, Foreign key (Students.id) | References the student who submitted the answer |

| question_id | Integer | Not Null, Foreign key (Questions.id) | References the question being answerd |
|---|---|---|---|
| answer_text | Text | Not Null | Student's submitted answer |
| submitted_at | Timestamp | Default Current Timestamp | Timestamp when the answer was first submitted |
| last_updated | Timestamp | Default Current Timestamp | Timestamp when the answer was first submitted |
| Unique constraint | - | Unique (student_id, question_id) | Ensures only one submission per student per question. |

## Scores Table

Table 6.4 Provides the Scores table, which maintains the evaluation outcomes of student submissions.

Table 6.4 Scores Table

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | Integer | Primary key, Auto increment | Unique score ID |
| submission_id | Integer | Not Null, Unique, Foreign key (Submissions.id) | Links to a specific submission |
| similarity_score | Real | - | Score based on similarity |
| rubric_json | Text | - | JSON object containing rubric evaluation |
| final_score | Real | - | AI-generated score based on rubric |
| teacher_score | Real | - | Editable score given by teacher manually |
| feedback | Text | - | AI-generated or teacher provided feedback |

## Key Features and Advantages of the Schema

- Ensures data integrity through primary and foreign key constraints.
- Prevents duplicate entries, such as multiple student records with the same roll number or multiple submissions for the same question.
- Supports efficient retrieval of information for evaluation, feedback, and reporting.
- Provides a structured framework that can scale as the number of students and questions increases.

This well-defined relational schema as depicted in Fig 6.1 enables the automated answer evaluation system to manage academic data efficiently, maintain consistency, and support seamless integration with the grading and frontend modules.

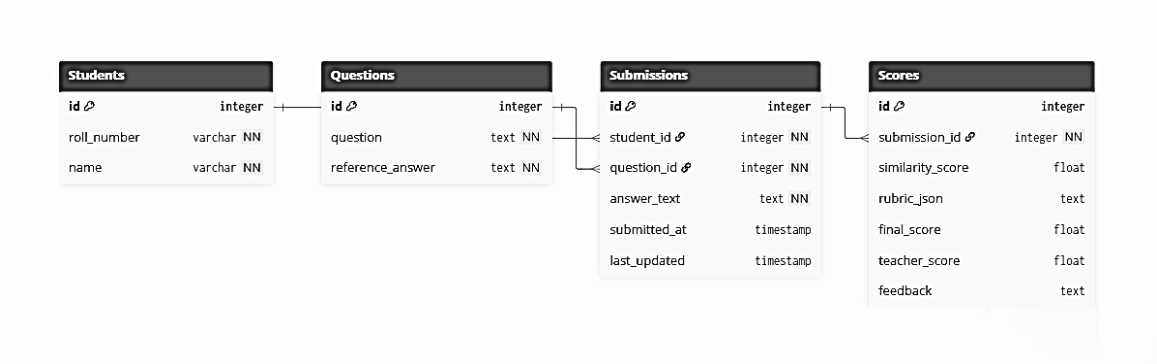


Fig 6.1 ER Diagram

## 6.3 User Interaction and Interface

The user interface is a critical component of the system, providing an intuitive and efficient means for teachers and students to interact with the backend modules. The frontend is implemented using Streamlit, a framework that offers simplicity, responsiveness, and seamless integration with Python. The application employs a sidebar-based navigation system, allowing users to easily switch between different functional sections, including adding questions, submitting answers, and viewing results. Input forms, dropdown menus, and expandable sections ensure that tasks can be performed with clarity and minimal effort. Real-time display of scores, rubric evaluations, and feedback enhances the overall user experience and provides immediate insights into student performance.

## End User Functionalities

The teacher or evaluator is the primary end user of the Automated Answer Evaluation System, and the interface has been designed to support all their assessment-related tasks in a simple and structured manner. The system offers the following key functionalities:

- **Student Management** – Teachers can add new students by entering a roll number and name. The system ensures uniqueness of each record to avoid duplication and maintains an organized student database for evaluation purposes.
- **Question Management** – Teachers can create and store new questions along with their corresponding reference answers. These reference answers form the basis for semantic similarity evaluation and rubric-based grading.
- **Submitting Answers** – Teachers can select a student, choose the relevant question, and enter the student's answer into the system. Once submitted, the response is automatically stored in the database and forwarded for evaluation.
- **Automated Evaluation** – Upon submission, the system computes semantic similarity between the student's response and the model answer using Sentence-BERT. It also requests rubric-based scoring and qualitative feedback from the Mistral 7B model. These results are then combined to produce a final AI-generated score.
- **Viewing Submissions and Results** – Teachers can review all student submissions in detail, including the original question, the student's answer, similarity scores, rubric evaluation, AI-generated feedback, and the final aggregated score.
- **Score Adjustment** – For cases where the AI-generated evaluation may require human judgment, teachers can override the score using a manual input option. The updated score and feedback are saved in the database for record-keeping and transparency.
- **Feedback Delivery** – The system provides structured, AI-generated feedback highlighting strengths, weaknesses, and suggestions for improvement. Teachers can use this directly or refine it before sharing with students.

Through these integrated functionalities, the system enables teachers to manage students, questions, submissions, and evaluations in one unified platform. The Streamlit-based frontend ensures a smooth and interactive user experience, bridging the backend processing with an intuitive interface that makes assessment both efficient and transparent.

## 6.4 Core Functional Modules

The core functional modules form the backbone of the automated answer evaluation system. They encapsulate the primary operations required to manage users, questions, submissions, and scoring. Each module is implemented in Python and interacts seamlessly with the SQLite database, ensuring that data is stored, retrieved, and processed efficiently. The modules are designed to separate concerns, making the system modular, maintainable, and scalable. The key functional modules are described in detail below

### Student Management

The student management module is responsible for maintaining records of all students interacting with the system. Its primary functions include

- **Adding Students** New students can be registered by providing a unique roll number and name. The system enforces uniqueness of roll numbers to prevent duplicate entries and maintain data integrity.
- **Retrieving Students** The module can fetch a list of all registered students, which is used in the frontend for dropdown selection during answer submission and result viewing.

This module ensures that all student-related information is consistently organized and easily accessible for other system functionalities.

### Question Management

The question management module handles the storage and retrieval of assessment questions and their corresponding reference answers. Its key functionalities are

- **Storing Questions** Teachers can add new questions with reference answers. The database schema enforces that each question is accompanied by a valid reference answer.
- **Retrieving Questions** The module can fetch all questions or a specific question by its ID. This functionality is essential for populating dropdowns in the frontend and for semantic similarity evaluation during grading.

By organizing questions and reference answers efficiently, this module provides the foundation for accurate and consistent evaluation of student responses.

### Submissions

The submissions module is responsible for linking students to their submitted answers while maintaining relevant metadata. Its functionalities include

- **Adding Submissions** When a student submits an answer, the system stores the submission with the student ID, question ID, answer text, and timestamps for creation and updates.
- **Retrieving Submissions** Submissions can be retrieved for a specific student or question, which is necessary for displaying results, calculating scores, and generating feedback.

The module also enforces a one-submission-per-student-per-question constraint, ensuring data integrity and simplifying score management.

### Scoring System

The scoring system module integrates the AI grading mechanisms and stores evaluation results. Its primary responsibilities are

**Storing Scores** After an answer is evaluated, the system saves multiple elements including

- **Similarity Score** Computed using SBERT to determine semantic alignment between the student answer and the reference answer.
- **Rubric JSON** A structured JSON object generated by the Mistral LLM that provides rubric-based scores for factual accuracy, completeness, clarity, and relevance.
- **AI Final Score** Aggregated score combining semantic similarity and rubric evaluation according to predefined weights.
- **Teacher Override** Optional manual adjustment of the final score by the teacher.
- **Feedback** Concise, structured feedback generated by the LLM highlighting strengths, weaknesses, and suggestions for improvement.

**Retrieving Scores** Stored scores are retrieved for display in the frontend, enabling both teachers and students to view results and feedback in an organized format.

## 6.5 Semantic Similarity and Rubric Evaluation

The grading mechanism of the system relies on two complementary approaches semantic similarity computation and rubric-based evaluation. These methods together enable a

nuanced assessment of student answers, capturing both content alignment and qualitative aspects such as clarity, completeness, and relevance.

## Similarity Computation

Semantic similarity is computed using Sentence-BERT (SBERT), a transformer-based model designed to generate dense vector embeddings for sentences that capture contextual and semantic meaning. This approach allows the system to go beyond mere keyword matching and identify the conceptual alignment between a student's answer and the reference answer. The process involves the following steps

- **Sentence Tokenization** Both the reference answer and the student's response are split into individual sentences. This enables fine-grained comparison and ensures that partial matches are captured accurately.
- **Embedding Generation** Each sentence is converted into a high-dimensional vector using the SBERT model. These embeddings encode semantic meaning in a way that allows similar sentences to have closer vector representations.
- **Cosine Similarity Calculation** A similarity matrix is constructed by calculating the cosine similarity between each reference sentence and each student sentence. The system identifies the most relevant matches for scoring and applies a weighting scheme to prioritize critical sentences in the reference answer.
- **Irrelevance Penalty** Sentences in the student answer that do not align with any reference sentences are penalized. This ensures that extraneous or off-topic content reduces the overall similarity score, promoting concise and focused responses.

This semantic similarity computation forms the quantitative foundation of the AI grading process, providing a reliable measure of content alignment.

## Rubric-Based Evaluation Using Mistral 7B

In addition to semantic scoring, the system integrates with the Mistral 7B Instruct LLM to perform qualitative rubric-based evaluation. The model is prompted with the question, reference answer, and student answer to assess multiple evaluation criteria, including

- **Factual Accuracy** Correctness of information in the student's response.
- **Completeness** Coverage of all essential points specified in the reference answer.
- **Clarity** Coherence and readability of the student's answer.
- **Relevance** Appropriateness of content in relation to the question.

The LLM returns structured JSON output containing rubric scores and concise feedback highlighting strengths, weaknesses, and suggestions for improvement. This feedback allows students to gain insights into areas needing attention while providing teachers with an interpretative evaluation alongside the semantic similarity score.

### Integration of Semantic and Rubric Scores

The final grading mechanism combines both semantic similarity and rubric-based evaluation into a single composite score, using predefined weightings to balance quantitative alignment with qualitative assessment. This ensures that student answers are scored not only based on content alignment but also on clarity, completeness, and overall presentation.

## 6.6 Implementation Screenshots

This section presents the essential interfaces of the automated answer evaluation system, illustrating the practical implementation and interaction between users and the system. The screenshots demonstrate the step-by-step workflow for managing questions, registering students, and submitting answers, reflecting the core functionalities of the system.

### Adding Questions

Fig 6.2 shows the teacher interface for adding questions with reference answers. The form requires both fields before submission, ensuring data completeness. Added questions appear in a list for easy review and management. This feature supports accurate semantic similarity and rubric-based grading, streamlining question bank setup and maintenance.

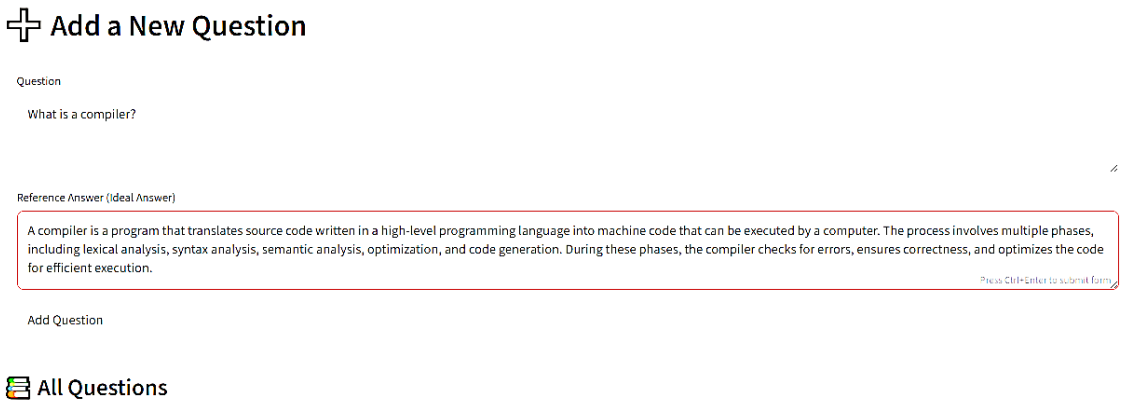


Fig 6.2 Add Question

## Adding Students

Fig 6.4 shows the student answer submission workflow. Teachers choose the student and question, input the response, and submit it for automated grading. The system combines Sentence-BERT for semantic analysis and Mistral 7B for rubric-based evaluation, ensuring accurate and efficient grading through an intuitive interface.

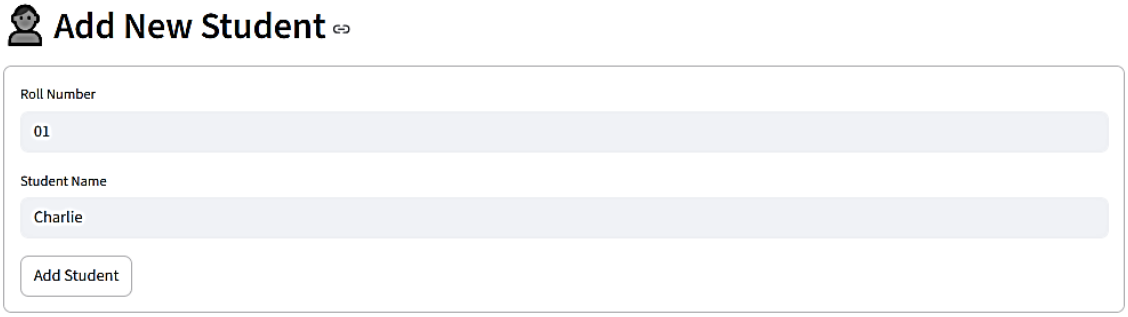


Fig 6.3 Add Student

## Submitting Answers

Fig 6.4 illustrates the workflow for submitting a student's answer. Teachers select the student and question, enter the response, and submit it for automated grading. The system uses Sentence-BERT for semantic similarity and Mistral 7B for rubric-based evaluation. This interface streamlines submission, ensures accuracy, and showcases the integration of AI-powered grading with a user-friendly design.

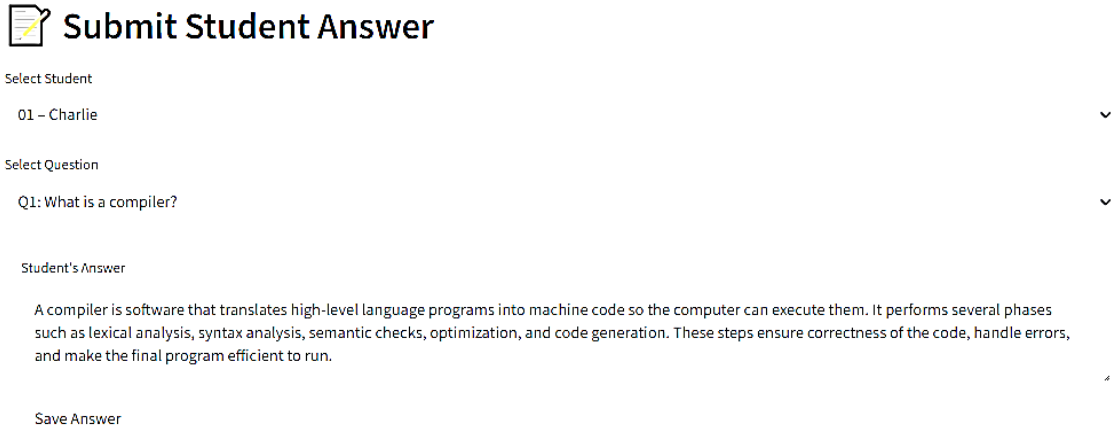


Fig 6.4 Submit Answer

# CHAPTER 7
# RESULTS AND DISCUSSION

## 7.1 Application Flow

The application flow of the automated answer evaluation system follows a sequential and structured process, ensuring that each student submission is accurately assessed and appropriately recorded. Initially, students submit their answers through the Streamlit frontend, where they select their identity and the corresponding question. Once submitted, the system stores the response in the SQLite database and triggers the grading pipeline, which involves computing semantic similarity scores between the student's answer and the reference answer. This step ensures that the alignment of content and context is quantitatively evaluated before any qualitative assessment.

Following the similarity computation, the system engages the Mistral 7B model to generate rubric-based scores and constructive feedback. These scores, combined with the similarity metrics, are aggregated into a final score that reflects both the semantic accuracy and the qualitative dimensions of the answer. Teachers can then review the results, provide optional overrides, and monitor student performance over time. This streamlined flow allows for consistent, reliable, and transparent evaluation, while maintaining the flexibility for human intervention where necessary.

## 7.2 Sample Test Cases

To evaluate the automated answer evaluation system, a representative set of test cases was selected to observe performance across varied response scenarios. These included highly accurate, partially correct, and irrelevant student answers, aiming to demonstrate core functionality and reliability in realistic situations. The system integrates semantic similarity analysis with rubric-based scoring to ensure fair and consistent results. Correct answers validate its ability to reward conceptual understanding, while partially correct ones reveal its feedback capabilities. Irrelevant responses highlight the penalty mechanism that discourages off-topic or filler content. Overall, these cases demonstrate the system's robustness and its potential as a practical tool for automated descriptive answer evaluation in education.

## High-Quality Answers

The first category, correct answers, includes responses that closely align with the reference answer in terms of content and structure. These cases help illustrate that the system can correctly identify accurate and complete answers, producing high similarity scores and positive rubric evaluations.

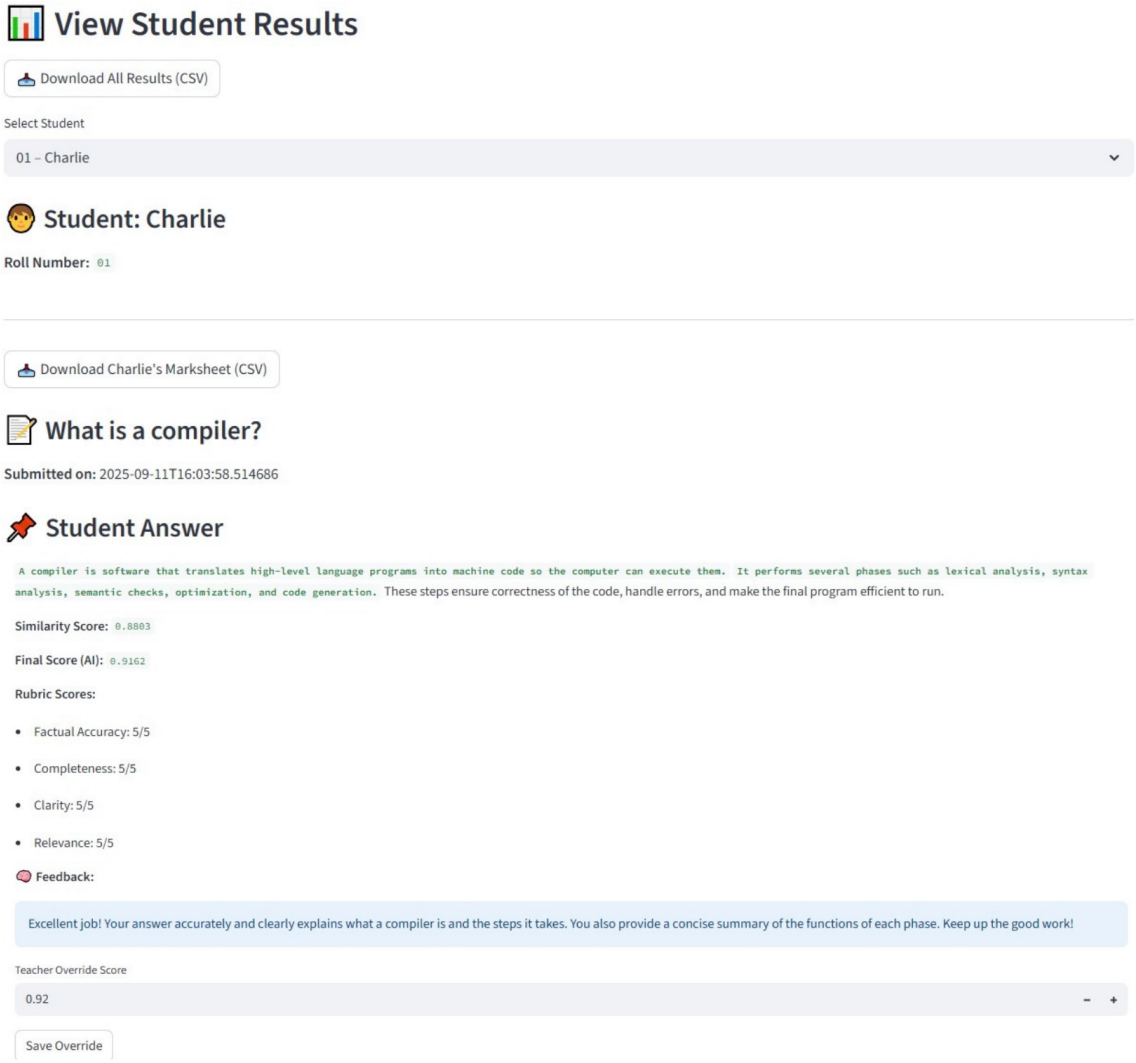


Fig 7.1 Highly correct answer result

## Partially Correct Answers

The second category, partially correct answers, comprises responses that convey some of the key concepts but omit certain points or lack clarity. These cases show how the system assigns intermediate scores and generates feedback highlighting areas for improvement.

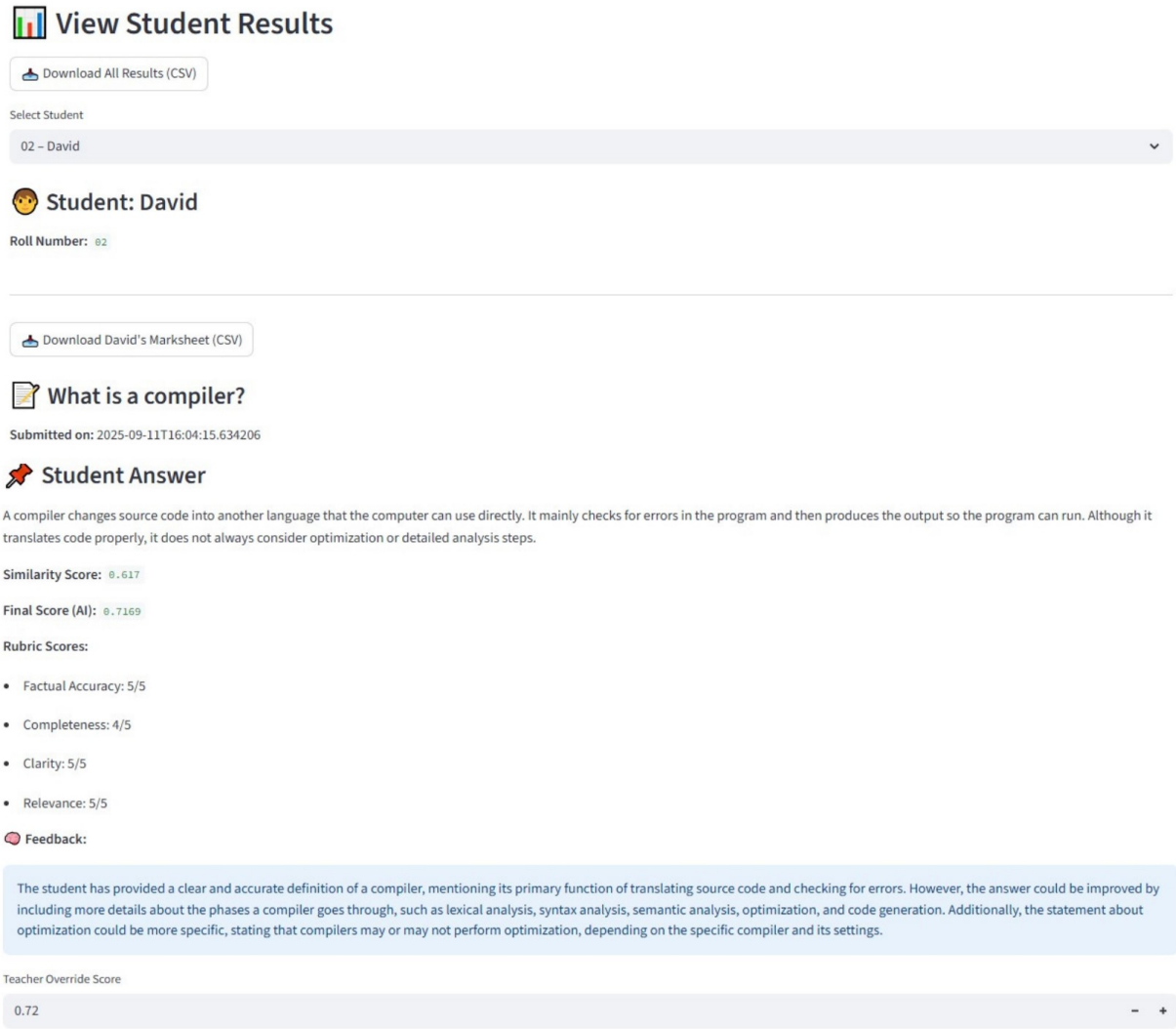


Fig 7.2 Partially correct answer result

## Irrelevant Answers

The final category, irrelevant or off-topic answers, contains content that is unrelated or incorrect, allowing observation of the irrelevance penalty mechanism and how the system avoids rewarding unrelated answers.

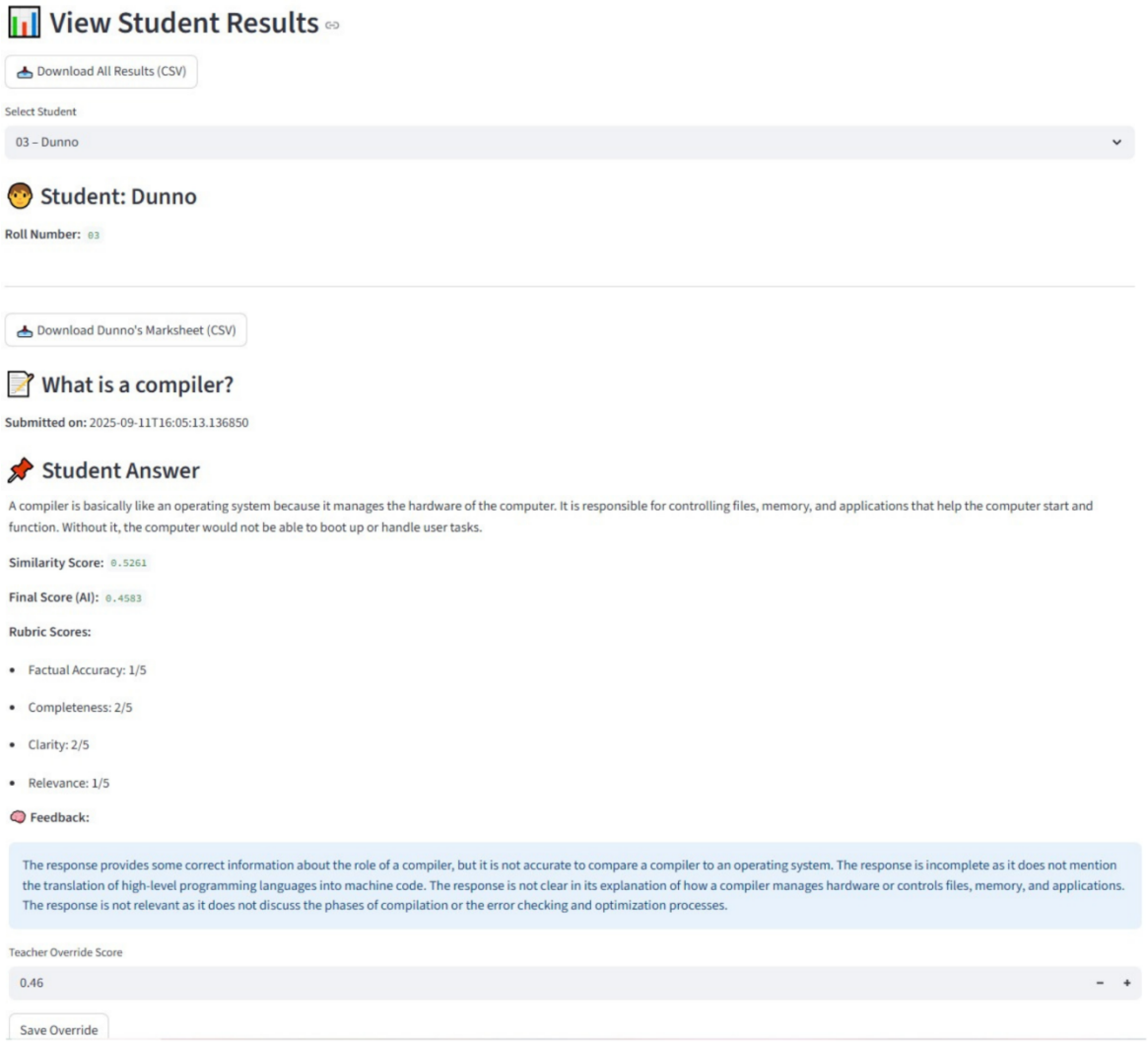


Fig 7.3 Irrelevant answer result

Overall, these sample test cases provide a snapshot of the system's performance, demonstrating its ability to combine semantic similarity scoring and rubric-based feedback to evaluate a range of student answers effectively.

## 7.3 Grading Results

The system calculates semantic similarity scores using Sentence-BERT embeddings and combines them with rubric-based scores generated by Mistral 7B. The results for each test case are summarized as follows

- **High-Quality Answers** High similarity scores (close to 1.0) and strong rubric scores, resulting in high final scores.
- **Partially Correct Answers** Moderate similarity and rubric scores, with final scores reflecting partial alignment with reference answers.
- **Irrelevant Answers** Low similarity scores; final scores are penalized due to off-topic content detection, despite rubric scores.

## 7.4 Feedback Results

The system generates feedback highlighting the strengths, weaknesses, and improvement suggestions for each answer

- **Strengths** Accurate points identified in student answers are recognized.
- **Weaknesses** Missing or incorrect information is highlighted.
- **Suggestions for Improvement** Clear guidance is provided for refining answers.

The feedback helps students understand their performance and guides them toward better conceptual clarity and presentation.

# CHAPTER 8
# CONCLUSIONS

## 8.1 Limitations of the Study

Despite its capabilities, the system has certain limitations. The accuracy of evaluation heavily relies on the quality and completeness of the reference answers, and highly creative or unconventional student responses may not always receive fair scores. The current irrelevance penalty mechanism is relatively basic and may not fully capture nuanced off-topic content. Additionally, depending on a single pre-trained language model and LLM for semantic similarity and rubric evaluation may restrict the system's effectiveness in handling domain-specific or highly technical content. The system also requires stable internet connectivity and consistent API access, which may impact availability and real-time performance.

## 8.2 Future Scope of Work

In the future, the system can be enhanced to improve both accuracy and applicability. Incorporating multiple pre-trained models or fine-tuning them for specific domains can improve semantic understanding and grading reliability. More sophisticated mechanisms for detecting irrelevant or off-topic content could make scoring fairer and more robust. Expanding the system to support additional answer types, such as long essays or coding solutions, could increase its versatility. Integration with learning management systems (LMS) could provide educators with real-time insights into student performance and learning trends. Furthermore, adaptive feedback mechanisms that recommend personalized learning resources based on identified weaknesses could further enhance the educational value of the system.

This project lays a solid foundation for AI-assisted assessment, showcasing both its advantages in efficient and consistent grading and the opportunities for future development to broaden its effectiveness in educational environments.

# References


1. US Patent US11967253B2, “Semi-automated evaluation of long answer exams.” *United States Patent and Trademark Office*.
2. US Patent US11967253B2, “Systems and methods for distilled BERT-based training model, for text classification” *United States Patent and Trademark Office*.
3. US Patent US11508251B2, “Method and system for intelligent identification and correction of questions” *United States Patent and Trademark Office*.
4. US Patent US5180309A, “Automated Answers Evaluation and Scoring System and Method” *United States Patent and Trademark Office*.
5. Gobrecht, A., Tuma, F., Möller, M., Zöller, T., Zakhvatkin, M., Wuttig, A., Sommerfeldt, H., & Schütt, S. (2024). “Beyond Human Subjectivity and Error: A Novel AI Grading System.” *IU International University of Applied Sciences.*
6. Yoon, S.-Y. (2023). “Short Answer Grading Using One-shot Prompting and Text Similarity Scoring Model.” *EduLab, Inc., Shibuya City, Tokyo, Japan.*
7. Saharan, R., Chauhan, R. K., Singh, S., & Sharma, P. (2020). “Automated Content Grading Using Machine Learning: Theoretical Content Grading from Exam Papers.” *Amity University, Uttar Pradesh, India*.
8. Reimers, N., & Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *Ubiquitous Knowledge Processing Lab, Department of Computer Science, Technische Universität Darmstadt*.